Worldline QMC: Code Structure

What each Python module does and why

Implementation notes

July 22, 2026

Abstract

A file-by-file account of the Python in `quantum_monte_carlo/`. The package `qmc/` is a conversion layer wrapped around the DSQSS/DLA worldline Monte Carlo engine (vendored in `external_dsqss/`, GPLv3, with one local patch); the top-level scripts are the command-line entry point, the correctness harness, and the benchmarks against the Chebyshev typicality code. The physics and the estimator corrections are documented separately in `qmc_implementation.tex`; this document is about the software.

Contents

1 Overview

1.1 Layout

```
quantum_monte_carlo/
|-- qmc/ the package: everything reusable
| |-- __init__.py re-exports the submodules
| |-- lattice.py lattice definition; single source of truth
| |-- parse.py readers/writers for DSQSS's own file formats
| |-- correlations.py estimator algebra: DSQSS output -> physics
| |-- storage.py HDF5 writer in the Chebyshev schema
| |-- console.py terminal styling for the run log
| `-- run.py orchestration; ties all of the above together
|-- install.sh builds the whole stack; ends in a smoke test
|-- submit_slurm.sh SLURM array-job template for parameter scans
|-- run_qmc.py command-line entry point
|-- validate_ed.py correctness harness (exact diagonalization)
|-- benchmark_vs_chebyshev.py benchmark at zero field, three sites
|-- benchmark_field.py benchmark at finite field, all four components
|-- plot_benchmark.py figures for the zero-field benchmark
`-- plot_benchmark_field.py figures for the finite-field benchmark
```

1.2 Dependency structure

The package is a shallow stack with no cycles. Only `run.py` knows about more than one concern:

Module	Imports from the package
`lattice.py`	— (numpy; `h5py` lazily, inside one method)
`parse.py`	—
`correlations.py`	—
`storage.py`	—
`console.py`	—
`run.py`	all five of the above

The four leaf modules are independently testable and contain no I/O beyond their own narrow file formats. That matters because the estimator corrections (Section 2.4) are the part most likely to need revisiting, and they live in a module with no dependencies at all.

1.3 Where the work actually happens

A single run passes through the package in this order:

1. `lattice.build` turns a spec string such as `square:6x4` into a `PeriodicLattice`.
2. `run.run` resolves parameters and calls `run.run_dsqss`.
3. `lattice.std_toml` renders the DSQSS input; `dla_pre` expands it into DSQSS's XML files.
4. `parse.write_site_displacements` writes a `disp.xml` covering only the requested displacement classes, and `run._set_param` points `param.in` at it.
5. `dla` runs; its progress is streamed through `run._echo_progress` and `console.Style`.
6. `parse.read_results` and `parse.read_displacement_kinds` read the raw output.
7. `correlations.*` converts the raw estimators into C^{xx}, C^{xy}, C^{zz}.
8. `storage.store` writes the HDF5 file in the Chebyshev schema.

2 The package

2.1 qmc/__init__.py — the package surface

Five lines. It re-exports the six submodules so that `import qmc` followed by `qmc.lattice.build(...)` works, and declares `__all__`. It contains no logic deliberately: importing the package should not have side effects, and in particular should not require the DSQSS binaries to be present. Scripts import what they need directly (`from qmc import run as run_mod`), so the re-export is a convenience rather than the expected entry route.

2.2 qmc/lattice.py — geometry and the single source of truth

Purpose. Defines the lattice and, critically, defines it *once* for both codes.

Why it is written this way. The Chebyshev code stores lattices as raw J_{ij} matrices with an arbitrary site ordering; some were built by hand with explicitly listed periodic pairs, and carry no geometric information at all. Recovering the Bravais structure from such a matrix in order to line up site indices with a QMC run is a graph-isomorphism problem in general. This module avoids that entirely by inverting the direction of information flow: one `PeriodicLattice` object emits *both* the DSQSS input and the Chebyshev-format coupling file, so the two codes agree on what "site 5" means by construction rather than by inspection.

Class `PeriodicLattice`.

Member	Role
`__init__(size, J, name)`	Stores `size` (per-dimension lengths), coupling `J`, and precomputes `coords`, the coordinate of every flat site index.
`_default_name()`	Produces `Chain_NN_PBC_N=8`, `Square_NN_PBC_N=24` etc., matching the Chebyshev naming so output filenames line up.
`_index_to_coord(i)`	Row-major, x-fastest unpacking, deliberately identical to DSQSS's `index2coord`: $i = x + L_x y + L_x L_y z$.
`_coord_to_index(c)`	The inverse.
`coupling_matrix()`	The J_{ij} matrix. Contributions are *accumulated* rather than assigned, which reproduces DSQSS's bond multiplicity: along a periodic direction of length 2 the $+1$ and -1 neighbours coincide and DSQSS lays down two bonds, giving an effective $2J$.
`displacement_vector(i,j)`	Minimum-image displacement, using the same wrapping convention as `dsqss.displacement`. Used to identify correlation bins.
`is_bipartite` (property)	True when every side length above 2 is even. Gates the antiferromagnet, which otherwise has a sign problem.
`marshall_sign(i,j)`	$(-1)^{\sum_d r_d}$. Undoes the sublattice gauge DSQSS uses to make the antiferromagnet sign-free; applied to transverse components only, and only for $J > 0$.
`neighbour_shells()`	Sites grouped by distance from site 0, nearest first, each entry carrying the representative site, its displacement, the members, and whether they are symmetry-equivalent. The last flag matters on anisotropic lattices, where $(1, 0)$ and $(0, 1)$ sit at equal distance but differ physically.
`shell_sites(shells)`	Translates n-th-neighbour indices into site indices, which is what `--sites` accepts. Raises naming the available range.

Member	Role
`write_coupling_file(path)`	Writes `all/J_ij` plus the `num_Spins` attribute — the Chebyshev `Couplings/` format. `h5py` is imported inside the method so the module stays importable without it.
`std_toml(...)`	Renders the DSQSS `std.toml`. Applies the Hamiltonian mapping $J_z = J_{xy} = -J$, $h = -h_z$.

Function `build(spec, J)`. Parses `chain:L`, `square:LxL`, `cube:LxLxL`. A single number expands over all dimensions (`square:4` → `4x4`); explicit forms may be anisotropic (`square:8x4`). Raises a clear `ValueError` on an unknown kind or a dimension-count mismatch, which `run_qmc.py` turns into a one-line CLI error.

2.3 `qmc/parse.py` — DSQSS's own file formats

Purpose. Everything that reads or writes a DSQSS-native file. This is the module that absorbs the engine's idiosyncrasies so nothing else has to.

Function	Role
`read_results(path)`	Parses `R <name> = <mean> <error>` lines, the format DSQSS uses for every observable in every output file. Returns {`name: (mean, error)`}.
`read_displacement_kinds(path)`	Reads `disp.xml`, mapping each ordered site pair to its displacement bin. Uses a regular expression rather than an XML parser because the file is not well-formed XML — it has bare comments and repeated top-level tags.
`write_site_displacements(...)`	Writes a `disp.xml` holding only the displacement classes actually requested. DSQSS's own generator enumerates all N^2 ordered pairs, making both the file and the per-step accumulator loop quadratic; a run needs a handful. Built by translating each site by the wanted displacement rather than scanning pairs, so it is $O(N \times n_{\text{sites}})$.
`collect_tau_series(...)`	Gathers `<prefix><kind>t<itau>` entries into `(nkinds, ntau)` arrays. Missing entries become NaN rather than zero, so a truncated or partial output file is visible instead of silently producing plausible-looking numbers.

Why the full Brillouin zone. DSQSS's own generator emits the product set $k_d \in \{0, \ldots, L_d/2\}$ and relies on $S(k) = S(-k)$ to cover the rest. That is a valid fundamental domain for $k \to -k$ in one dimension but *not* in two or more: on a 4×4 lattice the orbit $\{(1,3),(3,1)\}$ lies entirely outside it, so those k are never measured and the back-transform to real space silently loses their contribution. Measuring the whole zone makes the transform exact with uniform weight and removes the multiplicity bookkeeping. The bug this fixes produced a 261σ violation of an exact symmetry; see `qmc_implementation.tex`.

2.4 `qmc/correlations.py` — estimator algebra

Purpose. Converts what DSQSS measures into what is physically wanted. This is the smallest module and the most subtle; every function here exists because the naive reading of the DSQSS output is wrong in some specific way.

Object	Role
`CLASS_C_ROWS`	`("xx","xy","yx","zz")`. The row order every output file uses, matching symmetry class C of the Chebyshev `CorrelationTensor`.
`transverse_from_cf(...)`	$C^{xx} =$ `cf`$/2$. The stock worm estimator does not record whether the head is S^+ or S^-, so what it reports is the symmetrised combination $\frac{1}{2}[G^{+-} + G^{-+}]$, which is exactly $2C^{xx}$.
`_reverse_tau(a)`	Maps $i \mapsto (n_\tau - i) \bmod n_\tau$.
`xy_from_antisymmetric(...)`	$\operatorname{Im} C^{xy} = \frac{1}{2}\mathcal{R}$ `af`, where `af` is the signed histogram added by the engine patch. The τ reversal is required because the worm bins the head–tail time displacement with the opposite orientation; that is invisible in the symmetric channel (even about $\beta/2$) but negates this one.
(no C^{zz} entry)	C^{zz} needs no conversion: the patched real-space estimator reports it directly as `D<kind>t<itau>` and `run.convert` reads it through. Earlier versions inverse-Fourier-transformed the structure factor here, which was correct but $O(N^2)$.
`mirror_to_full_beta(...)`	Extends a $[0, \beta)$ series to $\tau = \beta$, with a sign flip for antisymmetric components.
`build_tensor(...)`	Allocates the zero-filled `(nsites, nrows, ntau)` arrays.

The suppressed warnings. numpy built against Apple's Accelerate BLAS leaves floating-point exception flags set inside its vectorised kernels, and numpy reports them after the call even when every input and output is finite — confirmed by feeding finite random input, where the warning fires while the result still matches `einsum` to 7×10^{-15}. Because the inputs are checked for finiteness immediately before and the outputs immediately after, the `errstate` block hides a known false alarm rather than a real condition.

2.5 `qmc/storage.py` — the output file

Purpose. Writes HDF5 in exactly the Chebyshev schema, so existing `Plot_*.py` scripts read QMC output unchanged.

Function	Role
`build_filename(...)`	Reproduces `Storage_Concept::create_file`'s naming rule. Optional fragments appear only when they differ from their defaults. `J` is an addition to the C++ scheme: there the couplings live in a named source file, whereas here J is a run parameter, and without it a ferromagnetic run would silently overwrite an antiferromagnetic one at the same β.
`_fmt(x)`	Formats like a C++ `ostream`: integers without a trailing `.0`, so `beta=2` not `beta=2.0`.
`MAX_TRIES`, `resolve_path(...)`	Collision avoidance, following the C++ writer: append `X` to the stem until a free name is found, up to four. Once all five variants exist the last is overwritten, so a repeated scan cannot fill the disk. (The C++ writer raises at that point instead.)

Function	Role
`store(...)`	Writes `/parameters` as attributes, the four `/results/*/<site>-0` datasets each with the same `info` attribute string as the C++ writer, and `/runtime_data`. Validates that the number of sites matches the array shape.

Every run writes four rows regardless of field. Both the transverse and longitudinal channels are measured on every run, so collapsing to a single row at $h_z = 0$ would discard the independent C^{zz} estimate — the cheapest available check on C^{xx}, since isotropy requires the two to agree.

2.6 `qmc/console.py` — terminal styling

Purpose. ANSI styling for the run log, and nothing else. Isolated in its own module so that presentation never entangles with computation.

`supports_colour(stream)` returns true only for a real terminal, and respects `NO_COLOR` and `TERM=dumb`. `Style` exposes semantic methods — `head`, `value`, `ok`, `warn`, `bad`, `path`, `bar` — which become no-ops when colour is unavailable.

Two design points are deliberate. Style is applied *after* padding, so column alignment is byte-identical with and without colour; and there is no faint/dim style, because it renders close to unreadable on terminals whose palette already has low contrast. Ordinary text keeps the terminal's default colour and emphasis is carried by bold and hue alone.

2.7 `qmc/run.py` — orchestration

Purpose. The only module that knows about all the others. It drives the external binaries, converts their output, and reports progress.

Locating the engine

`INSTALL_DIR` defaults to `dsqss_install/` beside the package and can be overridden with the `DSQSS_INSTALL` environment variable. `_binary(name)` raises a message naming the expected path rather than letting a bare `FileNotFoundError` escape.

Running the sampler

`run_dsqss` writes `std.toml`, invokes `dla_pre` to expand it, overwrites `wv.xml` with the full Brillouin zone, then runs `dla` (under `mpirun` when `ncores > 1`).

The subprocess is *streamed* rather than captured, so progress appears while the sampler runs instead of arriving in one lump at the end. The full log is still retained in memory so that a non-zero exit can be reported with context. `_set_param` edits `param.in` in place, which is how the trimmed `disp.xml` is substituted for the one `dla_pre` would have built: `std.toml` declares no `dispfile`, so the generator skips its own $O(N^2)$ enumeration entirely.

`_SET_DONE` and `_NCYC` are the regular expressions that recognise DSQSS's own progress chatter; `_echo_progress` reformats a matching line into a progress bar and returns whether it recognised it. `_format_seconds` renders durations as `45s`, `2m31s`, `1h04m`.

Seeding

`resolve_seed` draws from OS entropy when no seed is given. A fixed default made every run of the same parameters replay one chain, so repeating a run added nothing while the `X`-suffix naming presented it as a second result. `run_dsqss` returns the seed it used and `convert` records that, rather than resolving its own — otherwise a direct `convert()` call would store a seed that never reached the sampler.

Converting

`convert` is the heart of the module. It reads the displacement bins and both raw estimator files, applies the three corrections, and assembles the `(nsites, 4, ntau)` tensors. It detects whether the engine is patched by checking for the `A` entries; an unpatched build leaves C^{xy} at zero rather than producing something wrong. A structure factor that is partially populated raises immediately, since that would mean the full Brillouin zone was not measured.

Reporting

`print_banner` lists the parameters before sampling starts. It deliberately does *not* name the destination file, because the name is only settled at write time. `_warn_nonfinite_errors` flags error bars that came back NaN — DSQSS forms the error as $\sqrt{\langle x^2\rangle - \langle x\rangle^2}$, which with few sets on a barely fluctuating observable can go slightly negative through cancellation. The correlations are unaffected, so this warns rather than aborting. `_print_destination` reports the final path and any `X` suffix.

The entry point

`run(...)` is the function every script calls. It builds the lattice, validates the site list, rejects a frustrated antiferromagnet up front, runs, converts, resolves the output path, writes, and cleans up the scratch directory unless `keep_workdir` is set.

3 Top-level scripts

3.1 `run_qmc.py` — command-line entry point

A thin argparse wrapper over `run.run`, with options deliberately mirroring the Chebyshev executable (`--beta`, `--num_TimePoints`, sites, cores). It adds `--write-couplings` to emit the matching coupling file, and `--quiet` to reduce output to the bare path so that `path=$(run_qmc.py ... --quiet)` works in scripts.

Configuration errors (`ValueError`) are caught and printed as a single line with exit code 2; a traceback would add nothing for a mistyped lattice spec or an out-of-range site.

3.2 `install.sh` — building the stack

Fetches DSQSS at a pinned tag, applies the estimator patch, builds against a project-local venv, links the `dsqss` Python package into it, and finishes with a smoke test asserting $C^{xx}(0) = \frac{1}{4}$ exactly, $C^{xx} = C^{zz}$ at zero field, and the presence of all three channels. That test earned its place on the first run by catching a half-patched build.

The reset before patching is a hard reset *and* a clean: restoring tracked files alone leaves behind the files the patch adds, and the tree then looks patched to a file-existence check while the tracked edits are gone.

Two properties of the result constrain its use, and the script reports both. `dla` links dynamically against the MPI it was built with, so batch jobs must load the same module; and the generated helper scripts carry an absolute shebang to `venv/bin/python`, so the directory cannot be moved afterwards.

3.3 `submit_slurm.sh` — batch template

A parameter scan as an array job. The division of labour follows from the sampler: ranks are independent chains combined only at the end, so ranks within a task buy error bars and array tasks buy throughput. Seeds are drawn per run and offset per rank, so tasks are independent without intervention; the template sets `--seed` explicitly only to make a scan reproducible.

3.4 `validate_ed.py` — the correctness harness

Builds the Hamiltonian $H = \sum_{i<j} J_{ij}\, \mathbf{S}_i \cdot \mathbf{S}_j + h_z \sum_i S_i^z$ independently, diagonalises it densely, and evaluates

$$C^{ab}(\tau) = \frac{1}{Z} \sum_{n,m} e^{-\beta E_n} e^{\tau(E_n - E_m)} A_{nm} B_{mn},$$

then compares every component of the QMC output against it. This is the ground-truth test: it shares no code with the QMC path beyond the lattice definition.

Agreement is accepted if the deviation is within tolerance in units of σ *or* below a small absolute threshold, because $\tau = 0$ has a vanishing statistical error — it is fixed by an operator identity — and a pure σ test is meaningless there.

Two constraints are enforced up front: at most 14 sites (dense diagonalisation), and a bipartite lattice unless $J < 0$. The help text steers towards `square:3x3 --J=-1` when the C^{zz} path is being touched, because lattices with short sides make $k \to -k$ trivial and hide Brillouin-zone coverage bugs — an earlier suite of `square:4x2` and `cube:2x2x2` passed while C^{zz} was wrong.

3.5 `benchmark_vs_chebyshev.py` — zero-field benchmark

Compares against real Chebyshev output on the 24-site periodic square, at a size where exact diagonalisation is impossible (2^{24} states). The lattice is `square:6x4`, whose coupling matrix is byte-identical to `Couplings/Square_NN_PBC_N=24.hdf5`, so sites 0, 1 and 7 are the local, nearest-neighbour and next-nearest correlations.

The two codes use different τ conventions: Chebyshev stores `linspace(0, beta/2, num_TimePoints)` — its `Tmax` is $\beta/2$ — while QMC stores a uniform grid on $[0, \beta)$. The comparison is made at the QMC grid points inside $[0, \beta/2]$ with the dense reference interpolated onto them, so the noisy data is never interpolated. $\tau = 0$ is excluded from the pull statistic and reported separately as an absolute deviation.

`--skip-existing` reuses completed runs, so the comparison can be re-run without repeating the sampling.

3.6 `benchmark_field.py` — finite-field benchmark

The same lattice at $h_z \neq 0$. The field data is local only (site 0) but carries all four rows, so this exercises what vanishes at zero field: C^{xx} and C^{zz} separately, and $\mathrm{Im}\, C^{xy}$ — which is measurable only with the patched estimator and cannot be checked by exact diagonalisation at this size.

Three file-format quirks are handled. The field files predate the per-site group layout, so `results/Re_correlation` is a flat $(4, n_\tau)$ dataset rather than a group. Error datasets are named `Re_stds` in newer files and `Re_stddev` in older ones. The $h_z = 2$ set carries *no* uncertainties at all — those runs are reported as absolute agreement only, in a separate block, rather than inventing a reference error. `AVAILABLE` records which (h_z, J) combinations exist and at which β.

3.7 `plot_benchmark.py` and `plot_benchmark_field.py` — figures

The zero-field script produces one figure per coupling sign: correlations on top, pull underneath, one column per site, all inverse temperatures overlaid. The field script does the same per (h_z, J) with one column per component, and omits the pull panel where the reference stores no uncertainties. It also carries a `KNOWN_BAD` set naming the $h_z = 2$ reference files that a superseded version of the Chebyshev code wrote with the coupling signs swapped and C^{xy} inverted; those are corrected on load, as a pure relabelling with no fitted parameters, and the figure says so. Plotted against τ/β so every temperature shares the horizontal range $[0, \frac{1}{2}]$, which makes the approach to the symmetry point at $\beta/2$ line up across temperatures and lets the pull traces be read as flat bands. $\tau = 0$ is dropped from the pull panel for the same reason it is dropped from the statistic.

4 Conventions that recur

Several habits appear throughout and are worth stating once.

Missing data becomes NaN, never zero. `collect_tau_series` fills absent entries with NaN so that a truncated output file is visible. Several call sites then check explicitly and raise.

Checks bracket suppressed warnings. Where a warning is silenced, the inputs are validated immediately before and the outputs immediately after, so the suppression cannot hide a real condition.

Physics identities are used as tests. $C^{xx}(0) = \frac{1}{4}$, $C^{xx} = C^{zz}$ at $h_z = 0$, $C^{yx} = -C^{xy}$, and $C^{xy}(\beta/2) = 0$ all hold exactly and are checked rather than assumed. The isotropy identity in particular is size-independent, and is what exposed the Brillouin-zone bug at a system size where no exact reference exists.

Configuration errors are not tracebacks. Anything the user can mistype raises `ValueError` with a message naming the fix, which the CLI prints as one line with exit code 2.

Presentation is separable. Every reporting function takes `out` and `style`, and `progress` defaults to `False` in `run_dsqss`, so library use is silent and the harnesses stay readable.